Effective Claude Code

Patterns from building a platform in 5 days

Husain Al-Mohssen, PhD

Overview

- **Calibrating experience & expectations** — what's working, what's frustrating?
- The evidence: why naive AI use hurts experienced devs
- Context management & the priority quadrant
- Spec-driven development & verification patterns
- Multi-agent orchestration: what worked, what failed
- Key takeaways & discussion

The Uncertainty Is the Point

- No one has this figured out. Not us, not anyone.
- The imperfection is permanent, not temporary.
- Our job: build patterns anyway.

A field report, not a playbook.

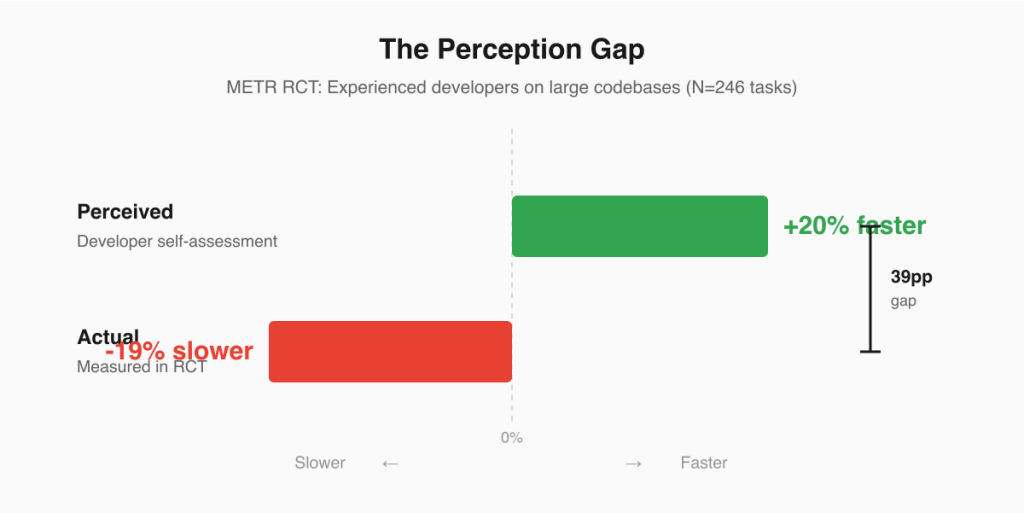
AI Can Hurt You

METR 2025 RCT: 16 experienced OSS developers

- **19% slower** with AI
- *Believed* they were **20% faster**
- **39-point** perception gap

Feb 2026 followup (57 devs): still no significant speedup.

Not anti-AI. Anti-**naive** adoption.



So What Does Work?

Not tips and tricks. **Structural patterns.**

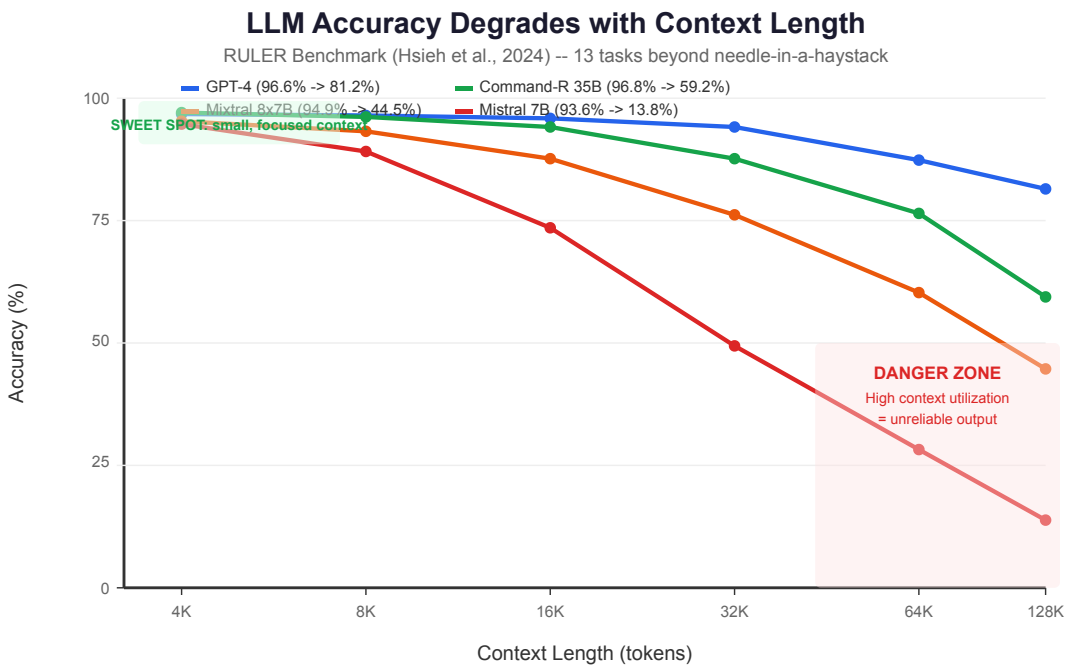
1. How the developer's role changes
2. How to manage agent context and execution
3. How to verify agent output
4. What failed

Context Is Everything

Model	4K	128K	Drop
GPT-4	96.6%	81.2%	-15 pts
Mixtral	94.9%	44.5%	-50 pts
Mistral 7B	93.6%	13.8%	-80 pts

Agent design = context design.

Each subagent gets the **minimum context** for its task. Fresh agent with 4K of focus beats a bloated session at 100K.



Context in Practice

Claude Code's `/context` command shows exactly where your tokens go.

- System prompt, tools, memory, skills, messages
- **Free space** is what matters — that's your agent's working memory
- When free space shrinks, accuracy drops
- Start a fresh session or subagent to reclaim it

```
/context
└ Context Usage
  claude-opus-4-6[1m] · 24k/1000k tokens (2%)
  Estimated usage by category
  System prompt: 6.2k tokens (0.6%)
  System tools: 8.5k tokens (0.9%)
  Memory files: 245 tokens (0.0%)
  Skills: 476 tokens (0.0%)
  Messages: 10.8k tokens (1.1%)
  Free space: 941k (94.1%)
  Autocompact buffer: 33k tokens (3.3%)

  Memory files · /memory
  └ ~/.claude/projects/-Users-husainal-mohssen-src-email-gateway/memory/MORY.md: 245 tokens

  Skills · /skills
```

Testability Determines Success

Anthropic C compiler case study:

Metric	Value
Agents	16 in parallel
Output	~100K lines of Rust
Cost	~$20K
Pass rate	**99%** GCC torture tests

Carlini: *"Most of my effort went into designing the tests, the environment, the feedback."*

The Testability Spectrum

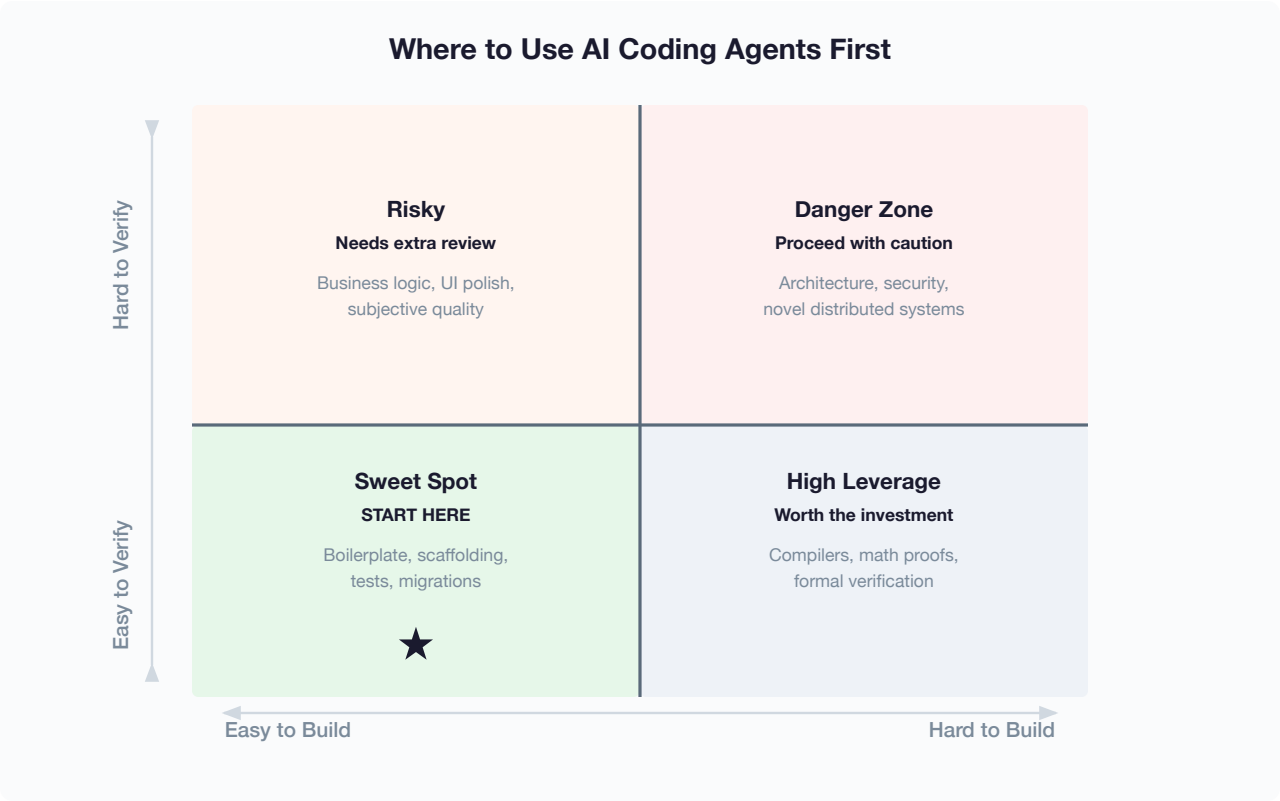
Domain	Testability	Agent effectiveness
Compilers	Binary correct/wrong	Excellent
Math proofs	Formal verification	Excellent
SWE-bench	Test suites	Good but incomplete
Web UI	Selenium/visual	Moderate
Business logic	Subjective	Weak without specs

The agent's capability is the ceiling. **The test infrastructure is the floor.**

Where to Use Agents First

Start bottom-left, earn your way right.

- **Sweet Spot:** Boilerplate, tests, migrations — easy to build, easy to verify
- **High Leverage:** Compilers, formal proofs — hard but verifiable
- **Risky:** Business logic, UI polish — easy to build, hard to verify
- **Danger Zone:** Architecture, security — hard on both axes

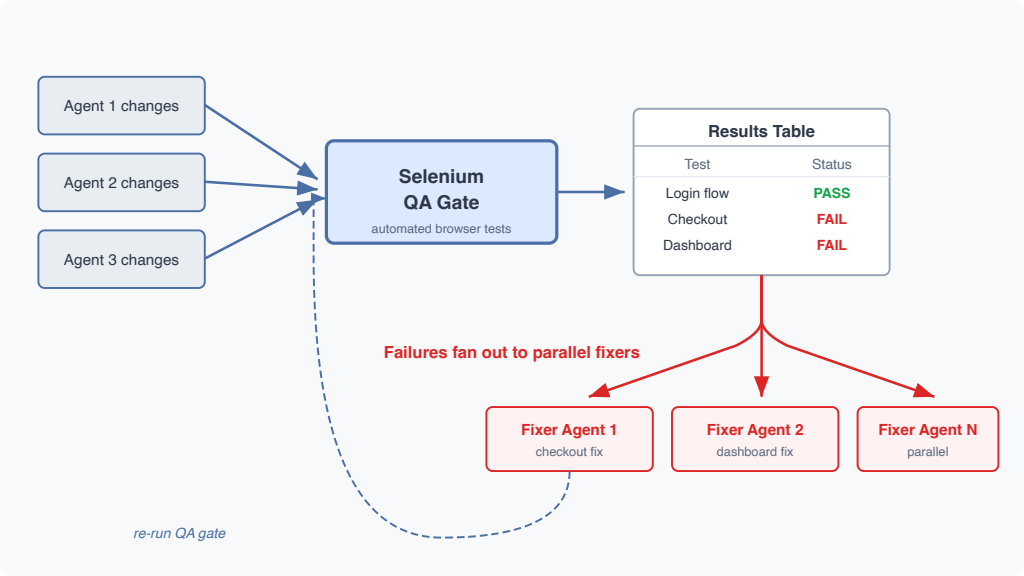


The QA Superpower

Multi-agent changes break the UI in ways unit tests miss.

- Caught avatar mismatch after multi-agent refactoring
- Caught stale refs, React state sync bugs
- Pass/fail tables enabled **parallel fix delegation**

Separated 7 phantom Chrome crashes from 2 real bugs.



Verification by Explanation

Force agents to **show their understanding**.

Technique	What it catches
Claim-by-claim table	Factual errors in your own descriptions
Comparison matrix	Hidden disagreements between agents
Accuracy score	Calibrated confidence before sharing
Diagram of understanding	Misunderstandings BEFORE they become bugs

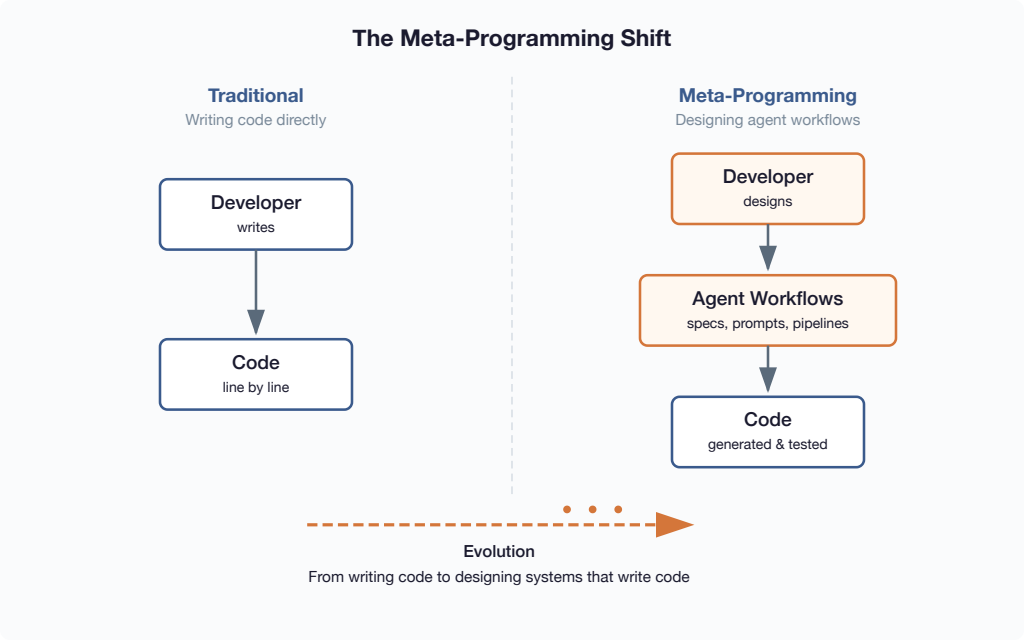
The diagram IS the verification. If the agent's picture doesn't match yours, its code won't either.

From Programmer to Meta-Programmer

You stop writing code. You start designing **agent workflows.**

1. **Design** the workflow
2. **Write** the spec
3. **Review** the output
4. **Iterate** on the spec, not the code

"dude have an agent do the work don't do it yourself"



Meta-Programming in Practice

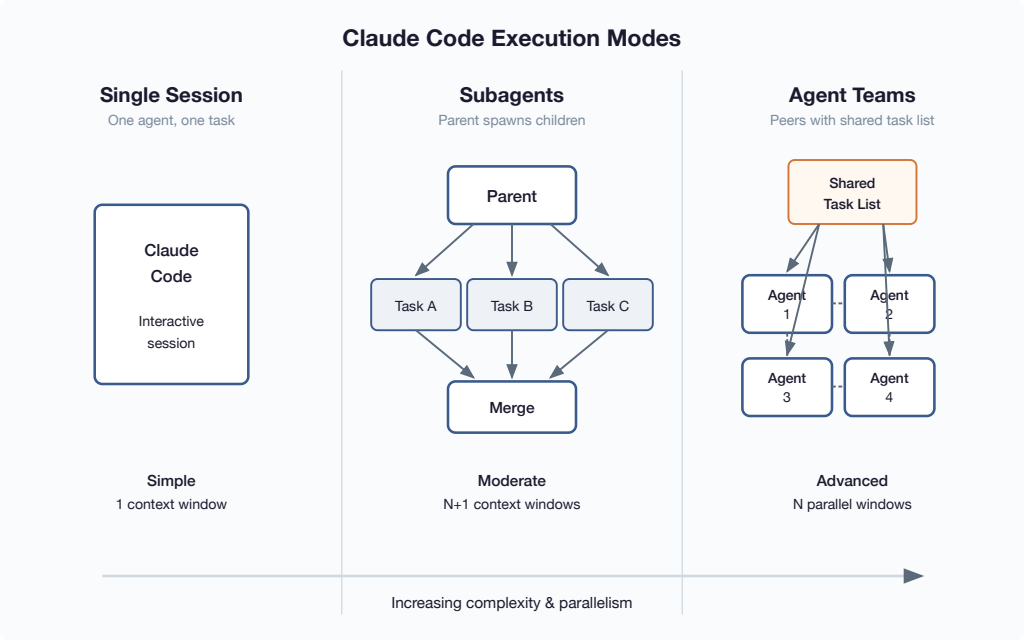
```
Day 1: "Claude, write a function that..."
Day 3: "Start 3 agents to each implement this spec,
        then start 2 auditors to cross-review"
Day 5: "12-agent, 5-phase pipeline with parallelism
        rules, gates, and selection criteria"
```

From **using AI to help me code** to **designing agent systems that produce code**.

44% of activity was still direct coding -- you never fully stop. The leverage is in the orchestration.

Know Your Execution Modes

Mode	What it is	When
Single session	One conversation, you drive	Simple tasks
Subagents	Spawned child processes	Parallel work
Agent teams	Shared task list + messaging	Large efforts (experimental)



Most patterns in this talk use **subagents**.

What to Say (and Not Say)

DO	DON'T
"Start 3 agents to each implement this spec"	"Help me write this function"
"All work by agents, you only supervise"	Let Claude do the work itself
"Have agent A audit agent B's output"	Ask one agent to check its own work
Give each agent a role and perspective	Dump everything into one session

Escalation: "start an agent" → "don't do this yourself" → "ALL agents not you" → "dude have an agent do it"

What Failed

- **Full autonomy is a myth** -- human intervention was constant
  - *"THERE IS NO WAY THEY DID A GREAT JOB"*
  - Rate limits killed all 3 builders in one round
- **Specs don't enforce themselves** -- 5/6 modules violated the frozen spec
- **Same-model auditors are lenient** -- they share blind spots
- **Quorum rules** -- never triggered, untested

Know When to Hold 'Em

> *"You got to know when to hold 'em, know when to fold 'em"*
> -- Kenny Rogers

The new killer SWE skill: **knowing when to review and when to trust.**

Trust it	Review it	Rewrite it
Boilerplate, scaffolding	Business logic, API contracts	Security-critical code
Tests pass + matches spec	Novel algorithm, edge cases	Anything you can't test

Key Takeaways

1. **Naive AI makes experienced devs slower** (METR 2025)
2. Your job is **meta-programming** -- design agent workflows, not code
3. **Context management** is the core skill -- smaller = more accurate
4. **Testability determines success** -- invest in test infrastructure first
5. **Force agents to show understanding** -- the diagram is the verification
6. **Know when to trust, when to review** -- that's engineering judgment now

Questions & Discussion

Husain Al-Mohssen

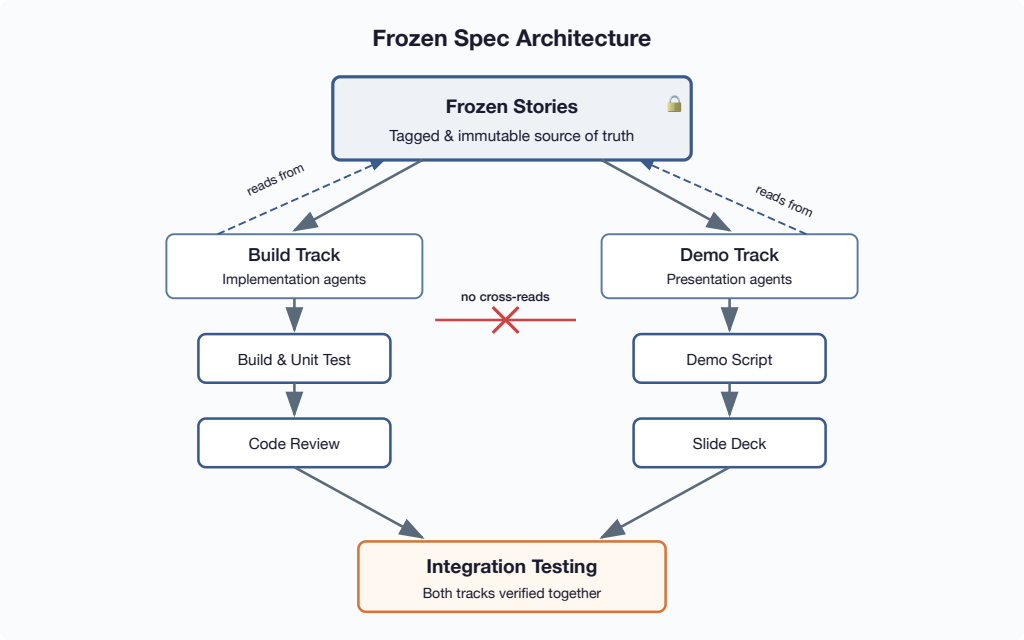
Appendix

Appendix: Frozen Specs Prevent Drift

Git-tagged, immutable specs as single source of truth.

- 16 stories frozen with `git tag -a e2e-stories-v1`
- 4 agents audited ~30 files, only 3 minor fixes
- Build and demo agents read from same frozen source

Limitation: Prevents spec-to-spec drift but NOT spec-to-code drift. 5/6 modules violated SKILL.md. Need active enforcement.



Appendix: Build → Test → Audit → Select

1. **Build** (parallel): N agents implement same spec
2. **Test** (sequential): automated suite against each
3. **Audit** (parallel): cross-review survivors
4. **Select**: pick the best, or none

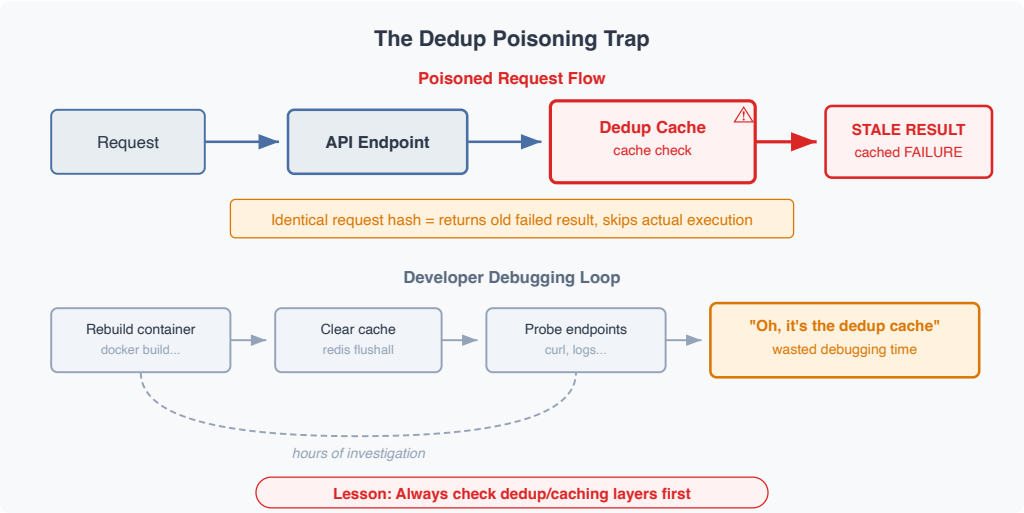
Cross-auditors caught 19 real issues. Cross-audit (builder A reviews builder C) beat separate auditor agents.

Appendix: Dedup Poisons Tests

1. API call fails
2. Response gets **cached**
3. Subsequent runs replay the **cached failure**
4. Tests look broken, API is fine

9 debugging steps before finding root cause. Hit twice. Blocked flagship feature.

Fix: Never cache errors. TTL-based expiry. No caching in test envs.



Appendix: Three-Agent Fact-Checking

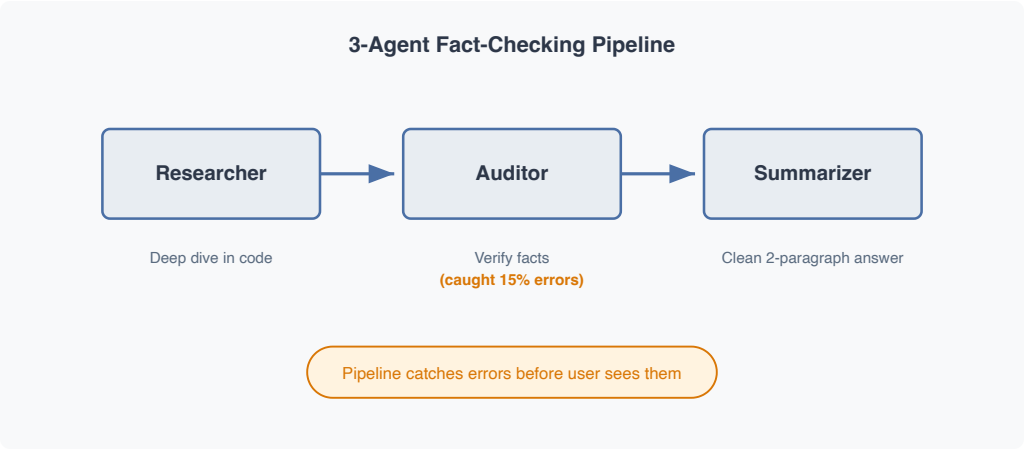
```
Researcher --> Auditor --> Summarizer
```

- 4/27 claims had errors (15% caught)
- Each agent sees only predecessor's output
- 3x latency (6-15 min vs 2-5 min)

Use when: sharing externally, making decisions.

Skip when: exploring, speed > accuracy.

Give auditors a **specific perspective** for better coverage.



Appendix: Ask for Alternatives

Before committing to any approach:

"Give me 3 ways to do this with trade-offs for each"

- Forces the model to explore the solution space, not just its first instinct
- You pick from a menu instead of accepting a default
- Works for architecture, API design, error handling, naming

Don't accept the first answer. Make the model compete with itself.

Appendix: Use the Advisor Sidekick

```
./advice.sh
```

Run it in a **separate terminal** alongside your Claude Code session.

- Coaching TUI that knows all the best practices in this talk
- Advises on prompting, agent orchestration, context management, verification
- **Never writes code or touches your files** — only advises
- Gets smarter as you add docs to `docs/`

Ask it things like:

- *"I have a 20-file refactor. How should I structure this?"*
- *"My session is getting long. What should I do?"*
- *"Should I use subagents here?"*

`github.com/mohsseha/effective_claude_code`

```
VERIFICATION & SKILL REFINEMENT WORKFLOW
========================================

┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 0 — Pipeline Health  (must pass before anything else)              │
│                                                                          │
│  AGENT 0                                                                 │
│  • Run run_all.sh, watch for exit 0 on all steps                         │
│  • Verify chat_indexer shows 0 inserted (all skipped → rowids stable)    │
│  • Verify embedder: 100% layer-1 hits, 0 API calls                       │
│  • Verify final DB counts unchanged from last run                        │
│  FAIL FAST — stop everything if any step errors                          │
└──────────────────────────────────────────────────────────────────────────┘
                          │ all steps exit 0
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 1 — Data & System Integrity  (5 agents in parallel)                │
│                                                                          │
│  AGENT 1                  AGENT 2                  AGENT 3               │
│  Data Completeness        Search Correctness       Cache Integrity       │
│  ─────────────────        ──────────────────       ───────────────       │
│  • message count in       • 10 FTS queries on      • vectors.db count    │
│    source DBs vs            known contacts           vs ingest_progress  │
│    chat_index.db          • 5 semantic queries     • 20 random chunks:   │
│  • doc/email count          verify chunk_text        source_rowid →      │
│    vs docs2md2_index        non-empty & sane         fts_content path    │
│  • no duplicate paths     • Arabic message           returns real text   │
│  • date range sane          search works             • layer-1 keys match  │
│                                                        current rowids    │
│  AGENT 4                  AGENT 5                                        │
│  Idempotency              Contact Resolution                             │
│  ────────────             ──────────────────                             │
│  • Run run_all.sh again   • Sample 50 messages                           │
│  • 0 inserted, 0 API        from group chats                             │
│    calls, identical         verify sender names                          │
│    row counts               are resolved (not                            │
│  • Row counts               raw phone numbers)                           │
│    exactly match          • Check % unresolved                           │
│    Phase 0 counts           (expect <5%)                                 │
└──────────────────────────────────────────────────────────────────────────┘
                          │ all agents report
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 2 — Integrator                                                     │
│                                                                          │
│  AGENT 6                                                                 │
│  • Collect Phase 1 reports, produce pass/fail per subsystem              │
│  • Block Phase 3 if any critical failure                                 │
│  • Log verified state + counts to project memory                         │
└──────────────────────────────────────────────────────────────────────────┘
                          │ no critical failures
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 3 — Multi-Model Skill Testing  (3 agents in parallel)              │
│                                                                          │
│  AGENT 7                  AGENT 8                  AGENT 9               │
│  claude-yolo              gemini-yolo              qwen-yolo             │
│  ────────────             ───────────              ─────────             │
│  Run 5 queries            Run same 5 queries       Run same 5 queries    │
│  across all 3 skills      across all 3 skills      across all 3 skills   │
│                                                                          │
│  Queries (same for all 3 models):                                        │
│  1. "find my last message to Meg"        → quick-chat-search             │
│  2. "find emails about my mortgage"      → quick-filesystem-search       │
│  3. "what documents do I have about      → quick-filesystem-search       │
│      health insurance"                                                   │
│  4. "what did Fadhel and I talk about    → quick-chat-search             │
│      recently"                                                           │
│  5. "find something conceptually         → quick-semantic-search         │
│      related to career planning"                                         │
│                                                                          │
│  Each agent records:                                                     │
│  • Did the skill get invoked correctly?                                  │
│  • Were results returned and sensible?                                   │
│  • What was confusing in the SKILL.md?                                   │
│  • How many tokens did the SKILL.md consume?                             │
│  • What failed or returned empty?                                        │
└──────────────────────────────────────────────────────────────────────────┘
                          │ all 3 models report
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 4 — SKILL.md Refinement  (3 agents in parallel, one per skill)     │
│                                                                          │
│  AGENT 10                 AGENT 11                 AGENT 12              │
│  quick-filesystem         quick-chat-search        quick-semantic        │
│  -search SKILL.md         SKILL.md                 -search SKILL.md      │
│  ─────────────────        ────────────             ────────────────      │
│  Using Phase 3 reports from all 3 models:                                │
│  • Remove redundant examples                                             │
│  • Tighten decision tables                                               │
│  • Fix anything that confused a model                                    │
│  • Cut tokens without losing correctness                                 │
│  • Re-test every example query after edits                               │
└──────────────────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PHASE 5 — Final Validation                                               │
│                                                                          │
│  AGENT 13                                                                │
│  • Re-run the same 5 queries on all 3 models with updated SKILL.mds      │
│  • Compare: did success rate hold or improve?                            │
│  • Compare: did token count go down?                                     │
│  • Write final report + update project memory                            │
└──────────────────────────────────────────────────────────────────────────┘

13 agents total across 6 phases.
```

Appendix: Session Details

Friday April 3, 9:30 am – 12:30 pm EDT · Organized by John Biasi (CTO, LineVision)

Pre-work (do before the session)

1. **Prerequisite:** At least 2 hours of real use with Claude Code
2. **Read:** How Coding Agents Work — Simon Willison (~15 min)
3. **Read:** Humans and Agents in SWE Loops — Martin Fowler (~15 min)
4. **Do:** Pick something you actually want built. Spend 1–2 hours building it with Claude Code. Come with notes on what worked, what surprised you, and what frustrated you.